# Linear Data Structures Toolkit (LDST)

## *School of Engineering & Technology (SOET)*

**Program:** B. Tech CSE (AI & ML), 2nd SEM

**Course Title:** Data Structure

**Course Code:** ETCCDS202

**Unit Title:** Linear Data Structures

**Submitted by:** Samarth Sharma

**Roll no:** 2501730279

**Section:** A

**Submission Date:** April 2026

# INTRODUCTION

In this assignment, we studied and implemented Linear Data Structures, which store elements in a sequential manner (one after another). These structures are very important in computer science because they help in organizing and managing data efficiently.

The main goal of this assignment was to build a **Linear Data Structures Toolkit (LDST)** in Python. In this toolkit, we implemented Dynamic Array, Singly Linked List, Doubly Linked List, Stack, and Queue from scratch. Along with this, we also solved a real-world problem using Stack, which is the Balanced Parentheses Checker.

# Dynamic Array

**Overview:**

A Dynamic Array is similar to a normal array, but it can grow in size when needed. In Python, lists work like dynamic arrays internally, but in this assignment, we implemented our own version to understand how resizing works.

**Implementation:**

- Initially, the array starts with a small capacity (for example, 2).
- When the array becomes full, a new array of double size is created.
- All existing elements are copied into the new array.
- The new element is then added.

**Operations Performed:**

- **append(x):** Adds an element at the end of the array.
- **pop():** Removes the last element and returns it.
- **print():** Displays the array along with its size and capacity.

**Amortized Time Complexity of append():-**

Most of the time, appending an element takes **O(1)** time because we simply add the element at the end.

However, when the array becomes full, resizing is required. During resizing:

- A new array is created.

- All elements are copied → takes **O(n)** time.

But resizing does not happen frequently. It happens only when the array is full. So, when we consider multiple operations together, the cost of resizing gets distributed.

Therefore, the **amortized time complexity of append() is O(1)**.

## Singly Linked List

**Overview:**

A Singly Linked List consists of nodes, where each node contains:

- Data
- A reference to the next node

Unlike arrays, linked lists do not store elements in contiguous memory locations.

**Operations Implemented:**

- **Insert at beginning(x):** Adds a node at the start → O(1)
- **Insert at end(x):** Adds a node at the end → O(n)
- **Delete by value(x):** Deletes the first occurrence of a value → O(n)
- **Traverse():** Prints all elements → O(n)

**Advantages:**

- No need to shift elements like arrays
- Dynamic in size
- Efficient insertion and deletion

**Limitations:**

- Cannot access elements directly (no indexing)
- Requires extra memory for storing pointers

## Doubly Linked List

**Overview:-**

A Doubly Linked List is an extension of a singly linked list where each node contains:

- Data
- Pointer to next node
- Pointer to previous node

This allows traversal in both forward and backward directions.

**Implementation:-**

- Nodes are connected using both next and previous pointers.
- The list maintains proper links during insertion and deletion.

**Operations Implemented:-**

- **Insert after node(target, x):** Inserts a node after a specific value → O(n)
- **Delete at position(pos):** Deletes node at a given position → O(n)

In this implementation, **0-based indexing** is used.

**Advantages**

- Easy traversal in both directions
- Easier deletion compared to singly linked list

**Limitations**

- Uses more memory due to extra pointer
- Slightly more complex implementation

## Stack (LIFO)

**Overview:-**

A Stack follows the **Last In First Out (LIFO)** principle. This means the last element inserted is the first one to be removed.

**Implementation:-**

The stack is implemented using a Singly Linked List:

- All operations are performed at the head for efficiency.

**Operations:-**

- **Push(x):** Inserts element → O(1)
- **Pop():** Removes top element → O(1)
- **Peek():** Returns top element → O(1)

**Time Complexity:-**

All operations (push, pop, peek) take constant time → **O(1)**

**Applications:-**

- Function call stack
- Undo/Redo operations
- Expression evaluation

## Queue (FIFO)

**Overview:-**

A Queue follows the **First In First Out (FIFO)** principle. This means the first element inserted is the first one to be removed.

**Implementation:-**

Queue is implemented using:

- Head pointer (for deletion)
- Tail pointer (for insertion)

**Operations:-**

- **Enqueue(x):** Adds element at rear → O(1)
- **Dequeue():** Removes element from front → O(1)
- **Front():** Returns front element → O(1)

**Time Complexity:-**

All operations (enqueue, dequeue, front) take constant time → **O(1)**

**Applications:-**

- Task scheduling
- Print queue
- BFS traversal

# Balanced Parentheses Checker

**Overview:-**

This is a practical application of Stack where we check whether a given expression has balanced parentheses.

**Approach:-**

- Traverse the string character by character
- If an opening bracket is found → push it onto the stack
- If a closing bracket is found → pop from stack and check if it matches
- At the end:
    - If stack is empty → Balanced
    - Otherwise → Not Balanced

**Supported Brackets:-**

- ()
- {}
- []

**Time Complexity:-**

O(n), where n is the length of the string

**Test Cases:-**

The following test cases were used to verify correctness:

| **Input** | **Output** |
|---|---|
| "([])" | True |
| "([)]" | False |
| "((((" | False |
| "" | True |

## Learning Outcomes:

From this assignment, I learned:

- How dynamic arrays resize internally
- Difference between arrays and linked lists
- Implementation of stack and queue using linked list
- Time complexity analysis in simple terms
- How stack can be used to solve real-world problems

## Conclusion:

This assignment helped in building a strong understanding of linear data structures. Implementing everything from scratch improved my logic building and problem-solving skills. The practical application of stack in checking balanced parentheses made the concepts more clear and meaningful.